

Interactive R tutorials for learning and assessments

Léo Belzile David A. Campbell Kyran Cupido
Nishan Mudalige Aurélien Nicosia Emily Somerset
Haochen Song David Riegert Russell Steele Irene Vrbik
Karen Huynh Wong

We review software tools for teaching, assessing and grading data analysis skills in the classroom using the **R** programming language.

Léo Belzile¹ , David A. Campbell² , Kyran Cupido³ , Nishan Mudalige⁴ , Aurélien Nicosia⁵, Emily Somerset⁴, Haochen Song⁴ , David Riegert⁶, Russell Steele⁷ , Irene Vrbik⁸, Karen Huynh Wong⁴

¹ HEC Montréal, leo.belzile@hec.ca

² Carleton University, davecampbell@math.carleton.ca

³ St. Francis Xavier University, kcupido@stfx.ca

⁴ University of Toronto, nishan.mudalige@utoronto.ca; emily.somerset@mail.utoronto.ca; fred.song@mail.utoronto.ca; karen.huynhwong@utoronto.ca

⁵ Université Laval, aurelien.nicosia@mat.ulaval.ca

⁶ Trent University, davidriegert@trentu.ca

⁷ McGill University, russell.steele@mcgill.ca

⁸ University of British Columbia, irene.vrbik@ubc.ca

Introduction

Data science majors popularity is at an all time high (Pierson 2023), leading to strain on teaching resources. The increased enrollment, even in upper level undergraduate courses, increases the time spent on class management by instructors and limits their ability and that of teaching assistants to provide timely feedback on evaluations. The advent of generative artificial intelligence (genAI) also raises concerns about plagiarism. Due to these, instructors more and more default back to invigilated multiple choice exams. This is unfortunate because the questions that can be asked in such evaluations are not necessarily authentic.

Data science is interdisciplinary by nature: standard tasks performed by analysts include exploratory data analysis and data wrangling, visualisation, model specification, estimation and validation, along with reporting. Designing assignments should therefore try to emulate those tasks while fostering statistical thinking (Woodard, Lee, and Woodard 2020). For formative and summative assessment, it is often beneficial to have clear and consistent grading rubrics, which facilitate learning experience (Poulos and Mahony 2008). Automating feedback by parsing code, and using natural language processing (NLP) for comments can lead to efficiency gains (Galassi and Vittorini 2021), while software solutions such as, e.g., Otter-Grader (Pyles and UC Berkeley Data Science Education Program 2025) facilitate sharing of grading rubrics, ensuring some consistency across instructors. Version control is now taught early on in introductory data science courses (e.g., Timbers, Campbell, and Lee 2022), and it is also becoming more frequent for class management (Rundel and Cetinkaya-Rundel 2025; Ricci, Medina, and Dogucu 2024). Solutions such as WebWork (Jahodova Berkova and Nemec 2020; Zimmerman 2020) or R-exams (Zeileis, Umlauf, and Leisch 2014) gives instructors tools to effectively manage large-scale assessments using randomized questions databases. Automation of many of these grading and feedback tasks remains difficult because, while there are wrong answers, there is no unique correct one. Thus, setting up rubrics requires much upfront work to determine most likely errors.

Our purpose is to review software tools for teaching, assessing and grading data analysis skills in the classroom using the **R** programming language (R Core Team 2025), a widely-used interpreted language which is popular among statisticians. There exists a wealth of contributed packages and software bundles to encourage semi-automation of tasks, generation of randomized evaluations, and tutorials. One key point is keeping the technical barriers low also for instructors, who can easily adapt their existing examples to a more interactive format. We consider several examples, notably:

- in-class activities with limited entry-level barriers to programming via [webR](#) coupled with [Quarto Live](#);
- interactive tutorials via the `learnR` (Aden-Buie et al. 2025; Stoudt, Scotina, and Luebke 2022a) package, including description of randomization within, with gradated hints and support for multiple solutions [gradethis](#) and recovery of student responses using [submitr](#) or [learnhash](#).
- semiautomatic tools for extracting student solutions for coding tasks and autograding, focusing on invariance of outputs.

The purpose of this paper is to review these three aspects critically with different points in mind, and to

- streamline use of these tools, by providing templates for users,
- review and highlight functionalities (e.g., limitations in `learnr` for randomization due to pre-rendering),
- suggest strategies for building unit tests and enabling informative feedback,

- report on instructors experience implementing these tools in their classroom during the 2025–2026 academic year.

WebR interface and interactive online tutorials

Description of tools

Tool	Description	Hosting	Related packages
webR	A version of R compiled for the browser and Node.js using WebAssembly. It allows R code to run in the user’s browser without an R server (r-wasm contributors 2026b).	Can work with static web hosting because computation happens client-side, although package support depends on WebAssembly-compatible R packages (r-wasm contributors 2026a, 2026b).	Useful for lightweight, browser-based R interactivity. It is conceptually different from standard learnr , which usually depends on Shiny.
learnr	An R package for turning R Markdown documents into interactive tutorials with narrative, code exercises, quizzes, videos, and Shiny components (Aden-Buie et al. 2025).	Standard learnr tutorials use <code>runtime: shiny_prerendered</code> , so they usually need to run locally or on a Shiny-capable hosting platform rather than as ordinary static pages (Aden-Buie et al. 2025).	Provides the tutorial framework that can incorporate gradethis feedback and, in some workflows, learnrhash tracking.
gradethis	An R package that provides automated feedback for student exercises in learnr tutorials. It can compare student code to a model solution or use custom grading logic (Aden-Buie et al. 2023).	Its deployment constraints generally follow learnr , since it is designed to run inside learnr tutorials (Aden-Buie et al. 2023).	Extends learnr by adding automated feedback to code exercises.

Tool	Description	Hosting	Related packages
Quarto	An open-source scientific and technical publishing system for documents, websites, blogs, books, presentations, dashboards, and other outputs (Posit Software, PBC 2026).	Can produce static HTML outputs that are suitable for static hosting; dynamic or live interactive content may require an appropriate runtime or hosting environment (Posit Software, PBC 2026).	Useful for publishing course materials, while learnr is more specialized for interactive tutorials.
Other tools?			

User case

Tool	Use case summary	Strengths	Limitations / comparison
webR	Use when students need to run R code directly in the browser without installing R or connecting to an R server (r-wasm contributors 2026b).	Strong for static or mostly-static websites because computation happens client-side. Useful for lightweight browser-based coding activities, demos, and interactive examples.	Less mature than standard server-side R workflows. The API is under active development, some packages may not work, and mobile browsers may have memory limits (r-wasm contributors 2026b). Compared with learnr , it is better for static hosting but less specialized for structured tutorials and feedback.

Tool	Use case summary	Strengths	Limitations / comparison
learnr	Use when you want a guided interactive tutorial with narrative, editable code exercises, quizzes, videos, and Shiny components (Aden-Buie et al. 2025).	Strong for self-paced active learning. Gives students a structured environment where they read, run code, answer questions, and receive feedback. Works well with gradethis .	Standard learnr tutorials use runtime: shiny_prerendered , so they usually need a Shiny-capable environment rather than ordinary static hosting (Aden-Buie et al. 2025). Compared with Quarto, it is more specialized for tutorials but less general for course websites, slides, books, or reports.
gradethis	Use when you want automated feedback on student code inside a learnr tutorial (Aden-Buie et al. 2023).	Strong for formative assessment. Can compare student code to a model solution or use custom grading logic, which helps give targeted feedback for common mistakes (Aden-Buie et al. 2023).	It is not a general publishing tool or a standalone assessment system. Its use is closely tied to learnr , so its hosting constraints usually follow learnr . Compared with learnr , it is not the tutorial framework itself; it is the feedback layer.

Tool	Use case summary	Strengths	Limitations / comparison
Quarto	Use when you want to create course notes, reports, slides, websites, blogs, books, dashboards, or other reproducible teaching materials (Posit Software, PBC 2026).	Strong for publishing. Supports multiple languages, including R, Python, Julia, and Observable, and can produce many output formats such as HTML, PDF, Word, slides, websites, and books (Posit Software, PBC 2026).	It is not primarily an interactive tutorial or grading system. Static outputs are easy to host, but live computation or rich interactivity may require additional tools or hosting. Compared with <code>learnr</code> , Quarto is broader and better for publishing; <code>learnr</code> is better for guided coding tutorials.

Interactive tutorials for learning and assessment using `learnr`

Main initial takeaways

- Best use is for interactive, low stakes tutorials.
- Supports active learning since students learn by running code.
- Good for formative assessment: useful for practice, readiness checks, lab prep, and review rather than formal high-stakes grading.
- Works best for smaller code tasks to simplify feedback through `gradethis`.
- Immediate feedback is available with `gradethis`.
- Student work can be tracked easily through `learnrhash`, as long as you need an existence proof and not a formal evaluation.
- The choice of hosting matters. Static tutorial `learnr` tutorials materials or pre-rendered R Markdown pages can be shared through GitHub Pages, but standard live interactive `learnr` tutorials require a Shiny-capable environment.

Some issues to think about:

- There are scoping issues which require careful planning when designing tutorials.
- Security is potentially an issue with regards to what you can do, particularly if you are serving the `learnr` tutorial from a server.
- It requires work to convert output from `learnr` hash into readable content for more formal grading.

- For questions where the next part of a question depends on the previous part, you should give the student the answer.
- `learnrhash` relies on Shiny server logic to collect the tutorial state and generate the copy-paste hash (`learnrhash` can not be implemented on GitHub pages as far as I am aware)

Autograding of code chunks from Rmarkdown and Quarto documents

Designing tutorials and assessment

In the following section, we focus on some elements that are idiosyncratic to **R**, although much of the discussion has similar implications for any other programming language.

- A simple way of getting around the cheating problem is through individualized data generation for which the procedure/script remains the same but answers differ. Of course, the drawback of this choice is that scripts could be shared between students. In data analysis problems, it is also possible to randomize the order of covariates, their name and their selection.
- Asking the right questions: determining the variables and setup (e.g., test statistic, one-sided versus two-sided test, confidence level, conclusion) that can be randomized.
- Generating context using generative AI.
- Validating the results (e.g., numerical tolerance to rounding, differences for p-values for one-sided versus two-sided).

Summative assessment of student answers

- Open-ended questions are best checked through multiple choice questions that force users to generate plots or look at summary statistics and model output.
- Focus on invariants and account for students degrees of freedom: e.g., in regression models, we can't check formulas because variable names may differ following data wrangling, the syntax and order of variables may be different (in **R** formula notation, `a*b` versus `b + a + a:b`). Likewise, contrast specification may change, as is the default engine class (e.g., `lm` vs `glm` with `family=gaussian`) for fitting these models. We can however compare sample sizes, span of projection (hat) matrices, linear contrasts, tests comparing nested models, residuals and predictions, etc.
- Depending on how prescriptive instructors are (for example, with naming of variables during tidying setup), we can validate more output.

Students could simply save certain objects as lists and submit a `.RData` or a CSV file with their short answer. Automated checks can verify class or attributes, and check existence of the variable names; settings may also be set so that the environment is saved at the end of each question. This requires that the answers be numerical, or TRUE/FALSE. If the inputs

are randomized, the instructor must run every data version to compare student answers with the solution key.

If students submit a Quarto file that generates an html/PDF report. consider chopping a Quarto file with headers defining questions and sub-questions to facilitate extraction of code or run a text parser to extract comments or text.

Evidence from the classroom



Caution

The list below has not been thoroughly checked and is not exhaustive.

learnr

Source	Evidence type	Classroom / teaching context	Summary
Stoudt, Scotina, & Luebke, “Supporting Statistics and Data Science Education with learnr” (Stoudt, Scotina, and Luebke 2022b)	Peer-reviewed education article	Statistics and data science education	Discusses classroom uses of learnr tutorials for statistics and data science instruction.
Engels, Grosjean, & Artus, “Teaching data science to students in biology using R, RStudio and Learnr” (Engels, Grosjean, and Artus 2023)	Peer-reviewed classroom study	Biology students learning data science	Analyzes three years of course data and describes learnr tutorials as part of the course design.
Aberson, “Building Interactive Tutorials for Teaching Psychological Statistics Online with learnr” (Aberson 2021)	Peer-reviewed teaching article	Psychological statistics	Describes building online interactive tutorials with videos, quizzes, and R analysis space using learnr .

Source	Evidence type	Classroom / teaching context	Summary
Stoudt, “Using learnr Tutorials in an Intro Stats Class as Pre-Labs” (Stoudt 2021)	Instructor classroom report / blog	Introductory statistics pre-labs	Practical classroom account of using learnr tutorials before lab sessions. Not peer-reviewed.

webR

Source	Evidence type	Classroom / teaching context	Summary
Perkel, “No installation required: how WebAssembly is changing scientific computing” (Perkel 2024)	Nature technology feature	Education-related use case	Discusses WebAssembly scientific computing and includes browser-based R as a way to reduce installation barriers. Not a classroom-outcomes study.
Rennie, “Teaching statistics interactively with webR” (Rennie 2023)	Conference talk / teaching demonstration	Statistics teaching	Demonstrates how webR can be used to make teaching materials more interactive with live coding examples. Not peer-reviewed.
White, “A brief introduction to using WebR and Quarto for client-side interactive lesson material” (White 2023)	Teaching blog / technical demonstration	Client-side interactive lesson materials	Shows how webR and Quarto can be used to let students run R code in lesson materials without installing R. Not peer-reviewed.
Ward, “stan-playground: Run Stan models directly in your browser” (Ward, Soules, and Magland 2026)	Peer-reviewed software paper	Browser-based statistical modeling for instruction	Not a classroom study of webR , but relevant to browser-based statistical computing for instruction; discusses integration with webR .

- Aberson, Christopher L. 2021. “Building Interactive Tutorials for Teaching Psychological Statistics Online with Learnr.” *Technology Innovations in Statistics Education* 13 (1). <https://doi.org/10.5070/T513153822>.
- Aden-Buie, Garrick, Daniel Chen, Garrett Grolemond, Alexander Rossell Hayes, and Barret Schloerke. 2023. *gradethis: Automated Feedback for Student Exercises in learnr Tutorials*. Posit Software, PBC. <https://pkgs.rstudio.com/gradethis/>.
- Aden-Buie, Garrick, Barret Schloerke, JJ Allaire, and Alexander Rossell Hayes. 2025. *learnr: Interactive Tutorials for R*. Posit Software, PBC. <https://doi.org/10.32614/CRAN.package.learnr>.
- Engels, Guyliann, Philippe Grosjean, and Frédérique Artus. 2023. “Teaching Data Science to Students in Biology Using r, RStudio and Learnr: Analysis of Three Years Data.” *Foundations of Data Science* 5 (2): 266–85. <https://doi.org/10.3934/fods.2022022>.
- Galassi, Alessandra, and Pierpaolo Vittorini. 2021. “Automated Feedback to Students in Data Science Assignments: Improved Implementation and Results.” In *Proceedings of the 14th Biannual Conference of the Italian SIGCHI Chapter*, 8. CHIItaly ’21 3. New York, NY, USA: Association for Computing Machinery. <https://doi.org/10.1145/3464385.3464387>.
- Jahodova Berkova, Andrea, and Radek Nemec. 2020. “Teaching Theory of Probability and Statistics During the Covid-19 Emergency.” *Symmetry* 12 (9): 1577. <https://doi.org/10.3390/sym12091577>.
- Perkel, Jeffrey M. 2024. “No Installation Required: How WebAssembly Is Changing Scientific Computing.” *Nature*. <https://doi.org/10.1038/d41586-024-00725-1>.
- Pierson, Steve. 2023. “Data Analytics, Data Science Degrees See Large Increases in 2022.” *AMStat News*, no. 558: 16–21. <https://magazine.amstat.org/blog/2023/12/01/degreesstats2022/>.
- Posit Software, PBC. 2026. *Quarto: An Open-Source Scientific and Technical Publishing System*. Posit Software, PBC. <https://quarto.org/>.
- Poulos, Ann, and Mary Jane Mahony. 2008. “Effectiveness of Feedback: The Students’ Perspective.” *Assessment & Evaluation in Higher Education* 33 (2): 143–54. <https://doi.org/10.1080/02602930601127869>.
- Pyles, Christopher, and UC Berkeley Data Science Education Program. 2025. “Otter-Grader: A Python and R Autograding Solution.” <https://doi.org/10.5281/zenodo.5259955>.
- R Core Team. 2025. *R: A Language and Environment for Statistical Computing*. Vienna, Austria: R Foundation for Statistical Computing. <https://www.R-project.org/>.
- Rennie, Nicola. 2023. “Teaching Statistics Interactively with webR.” Talk at the RSS International Conference 2023. <https://nrennie.rbind.io/talks/rss-conference-teaching-webR/>.
- Ricci, Federica Zoe, Catalina Mari Medina, and Mine Dogucu. 2024. “Automated Grading Workflows for Providing Personalized Feedback to Open-Ended Data Science Assignments.” *Technology Innovations in Statistics Education* 15 (1). <https://doi.org/10.5070/t5.1886>.
- Rundel, Colin, and Mine Cetinkaya-Rundel. 2025. *ghclass: Tools for Managing Classes on GitHub*. <https://github.com/rundel/ghclass>.
- r-wasm contributors. 2026a. *Installing R Packages for webR*. r-wasm. <https://docs.r-wasm.org/webR/latest/packages.html>.
- . 2026b. *webR: R in the Browser*. r-wasm. <https://docs.r-wasm.org/webR/latest/>.

- Stoudt, Sara. 2021. “Using Learnr Tutorials in an Intro Stats Class as Pre-Labs.” Blog post. <https://sastoudt.github.io/posts/2021-06-05-learnr-tutorials-intro-stat/>.
- Stoudt, Sara, Scotina Anthony D., and Karsten Luebke. 2022a. “Supporting Statistics and Data Science Education with `learnr`.” *Technology Innovations in Statistics Education* 14 (1). <https://doi.org/10.5070/t514156264>.
- Stoudt, Sara, Anthony D. Scotina, and Karsten Luebke. 2022b. “Supporting Statistics and Data Science Education with Learnr.” *Technology Innovations in Statistics Education* 14 (1). <https://doi.org/10.5070/T514156264>.
- Timbers, T., T. Campbell, and M. Lee. 2022. *Data Science: A First Introduction*. Chapman & Hall/CRC Data Science Series. CRC Press. <https://www.routledge.com/Data-Science-A-First-Introduction/Timbers-Campbell-Lee/p/book/9780367524685>.
- Ward, Brian, Jeff Soules, and Jeremy Magland. 2026. “stan-playground: Run Stan Models Directly in Your Browser.” *Journal of Open Source Software* 11 (118): 9531. <https://doi.org/10.21105/joss.09531>.
- White, Ethan P. 2023. “A Brief Introduction to Using WebR and Quarto for Client-Side Interactive Lesson Material.” Blog post. <https://jabberwocky.weecology.org/2023/03/13/a-brief-introduction-to-using-webr-and-quarto-for-client-side-interactive-lesson-material/>.
- Woodard, Victoria, Hollylynne Lee, and Roger Woodard. 2020. “Writing Assignments to Assess Statistical Thinking.” *Journal of Statistics Education* 28 (1): 32–44. <https://doi.org/10.1080/10691898.2019.1696257>.
- Zeileis, Achim, Nikolaus Umlauf, and Friedrich Leisch. 2014. “Flexible Generation of e-Learning Exams in R: Moodle Quizzes, OLAT Assessments, and Beyond.” *Journal of Statistical Software* 58 (1): 1–36. <https://doi.org/10.18637/jss.v058.i01>.
- Zimmerman, Jane K. 2020. “Implementing Standards-Based Grading in Large Courses Across Multiple Sections.” *PRIMUS: Problems, Resources, and Issues in Mathematics Undergraduate Studies* 30 (8–10): 1040–53. <https://doi.org/10.1080/10511970.2020.1733149>.